



PRACTICAL - 7

Implementation of Making Change Problem using Dynamic Programming

Aim of the Practical:

To find the minimum number of coins required to make a target amount using dynamic programming, display every decision clearly, reconstruct the selected coins, and analyze complexity.

Algorithm:

1. Set $dp[0] = 0$ and all other entries to infinity. 2. For each amount x from 1 to A , test every coin. 3. If the coin can be used, compare $dp[x]$ with $dp[x - \text{coin}] + 1$. 4. Store both the minimum count and the coin that produced it. 5. Follow the stored coins backward to reconstruct the solution.

Program:

```
#include <limits>
#include <iomanip>
#include <iostream>
#include <vector>
using namespace std;

int main() {
    vector<int> coins = {1, 3, 4};
    int amount = 6;
    vector<int> dp(amount + 1, INT_MAX), chosen(amount + 1, -1);
    dp[0] = 0;

    cout << "MAKING CHANGE USING DYNAMIC PROGRAMMING\n";
    cout << "Coins = {1, 3, 4}, Target amount = 6\n";
    cout << "dp[0] = 0; all other entries start as INF\n\n";
    cout << "STEP-BY-STEP CALCULATION\n";
    for (int x = 1; x <= amount; ++x) {
        cout << "\nAmount " << x << ":\n";
        for (int coin : coins) {
            cout << " Coin " << coin << ": ";
            if (coin > x) { cout << "cannot use (coin > amount)\n"; continue; }
            int candidate = dp[x - coin] == INT_MAX
                ? INT_MAX : dp[x - coin] + 1;
            cout << "dp[" << x - coin << "] + 1 = "
                << dp[x - coin] << " + 1 = " << candidate;
            if (candidate < dp[x]) {
                dp[x] = candidate; chosen[x] = coin;
                cout << " -> update dp[" << x << "] = " << candidate
                    << ", coin[" << x << "] = " << coin;
            } else cout << " -> no change";
            cout << '\n';
        }
        cout << " Final: dp[" << x << "] = " << dp[x]
            << ", chosen coin = " << chosen[x] << '\n';
    }

    cout << "\nDP TABLE (minimum coins)\nAmount\n";
    for (int x=0; x<=amount; ++x) cout << setw(4) << x;
    cout << "\ndp[x]\n";
    for (int x=0; x<=amount; ++x) cout << setw(4) << dp[x];
    cout << "\nCHOICE TABLE (last coin selected)\nAmount\n";
    for (int x=0; x<=amount; ++x) cout << setw(4) << x;
    cout << "\ncoin[x]\n";
    for (int x=0; x<=amount; ++x) {
        if(chosen[x]<0) cout<<setw(4)<<"-";
        else cout<<setw(4)<<chosen[x];
    }

    cout << "\n\nRECONSTRUCTION\n";
    int remaining=amount, step=1;
    while(remaining>0){
        int coin=chosen[remaining];
        cout << "Step " << step++ << ": amount " << remaining
            << " uses coin " << coin << "; remaining = " << remaining
            << " - " << coin << " = " << remaining-coin << '\n';
        remaining-=coin;
    }
```

```

    }
    cout << "Selected coins: {3, 3}\nTotal value: 3 + 3 = 6\nMinimum number of coins: " << dp[amount] << '\n';
    cout << "Time complexity: Theta(A*m)\nSpace complexity: Theta(A)\n";
}

```

Output:

MAKING CHANGE USING DYNAMIC PROGRAMMING
 Coins = {1, 3, 4}, Target amount = 6
 dp[0] = 0; all other entries start as INF

STEP-BY-STEP CALCULATION

Amount 1:

Coin 1: $dp[0] + 1 = 0 + 1 = 1$ -> update $dp[1] = 1$, coin[1] = 1
 Coin 3: cannot use (coin > amount)
 Coin 4: cannot use (coin > amount)
 Final: $dp[1] = 1$, chosen coin = 1

Amount 2:

Coin 1: $dp[1] + 1 = 1 + 1 = 2$ -> update $dp[2] = 2$, coin[2] = 1
 Coin 3: cannot use (coin > amount)
 Coin 4: cannot use (coin > amount)
 Final: $dp[2] = 2$, chosen coin = 1

Amount 3:

Coin 1: $dp[2] + 1 = 2 + 1 = 3$ -> update $dp[3] = 3$, coin[3] = 1
 Coin 3: $dp[0] + 1 = 0 + 1 = 1$ -> update $dp[3] = 1$, coin[3] = 3
 Coin 4: cannot use (coin > amount)
 Final: $dp[3] = 1$, chosen coin = 3

Amount 4:

Coin 1: $dp[3] + 1 = 1 + 1 = 2$ -> update $dp[4] = 2$, coin[4] = 1
 Coin 3: $dp[1] + 1 = 1 + 1 = 2$ -> no change
 Coin 4: $dp[0] + 1 = 0 + 1 = 1$ -> update $dp[4] = 1$, coin[4] = 4
 Final: $dp[4] = 1$, chosen coin = 4

Amount 5:

Coin 1: $dp[4] + 1 = 1 + 1 = 2$ -> update $dp[5] = 2$, coin[5] = 1
 Coin 3: $dp[2] + 1 = 2 + 1 = 3$ -> no change
 Coin 4: $dp[1] + 1 = 1 + 1 = 2$ -> no change
 Final: $dp[5] = 2$, chosen coin = 1

Amount 6:

Coin 1: $dp[5] + 1 = 2 + 1 = 3$ -> update $dp[6] = 3$, coin[6] = 1
 Coin 3: $dp[3] + 1 = 1 + 1 = 2$ -> update $dp[6] = 2$, coin[6] = 3
 Coin 4: $dp[2] + 1 = 2 + 1 = 3$ -> no change
 Final: $dp[6] = 2$, chosen coin = 3

DP TABLE (minimum coins)

Amount	0	1	2	3	4	5	6
dp[x]	0	1	2	1	1	2	2

CHOICE TABLE (last coin selected)

Amount	0	1	2	3	4	5	6
coin[x]	-	1	1	3	4	1	3

RECONSTRUCTION

Step 1: amount 6 uses coin 3; remaining = $6 - 3 = 3$
 Step 2: amount 3 uses coin 3; remaining = $3 - 3 = 0$
 Selected coins: {3, 3}
 Total value: $3 + 3 = 6$
 Minimum number of coins: 2
 Time complexity: $\Theta(A \cdot m)$
 Space complexity: $\Theta(A)$

DP Table (Minimum Number of Coins):

Amount	0	1	2
dp[x]	0	1	2
Amount	3	4	5
dp[x]	1	1	2
Amount	6		
dp[x]	2		

Choice Table (Last Coin Selected):

Amount	0	1	2
coin[x]	-	1	1
Amount	3	4	5
coin[x]	3	4	1
Amount	6		
coin[x]	3		

Table Interpretation:

dp[x] records the minimum number of coins needed for amount x. coin[x] records the last coin chosen for that optimal value. For amount 6, coin[6] = 3. Moving to amount 3 gives coin[3] = 3, and moving to amount 0 completes the reconstruction.

Reconstruction of Selected Coins:

Step 1: amount 6 uses coin 3; remaining amount = $6 - 3 = 3$.

Step 2: amount 3 uses coin 3; remaining amount = $3 - 3 = 0$.

Therefore, the selected coins are {3,3}; their value is $3 + 3 = 6$ and the minimum number of coins is 2.

Time Analysis:

Let A be the target amount and m be the number of coin denominations. The algorithm evaluates all m coins for every amount from 1 to A.

$$T(A,m) = A \times m \times O(1) = \text{Theta}(A*m)$$

$$S(A) = \text{Theta}(A)$$

Final Time Analysis:

- Time Complexity: $\text{Theta}(A*m)$.

- Space Complexity: $\text{Theta}(A)$.

Result:

The Making Change problem was solved successfully using dynamic programming. For coins {1,3,4} and amount 6, the tables and reconstruction show the optimal solution {3,3}, requiring 2 coins.